

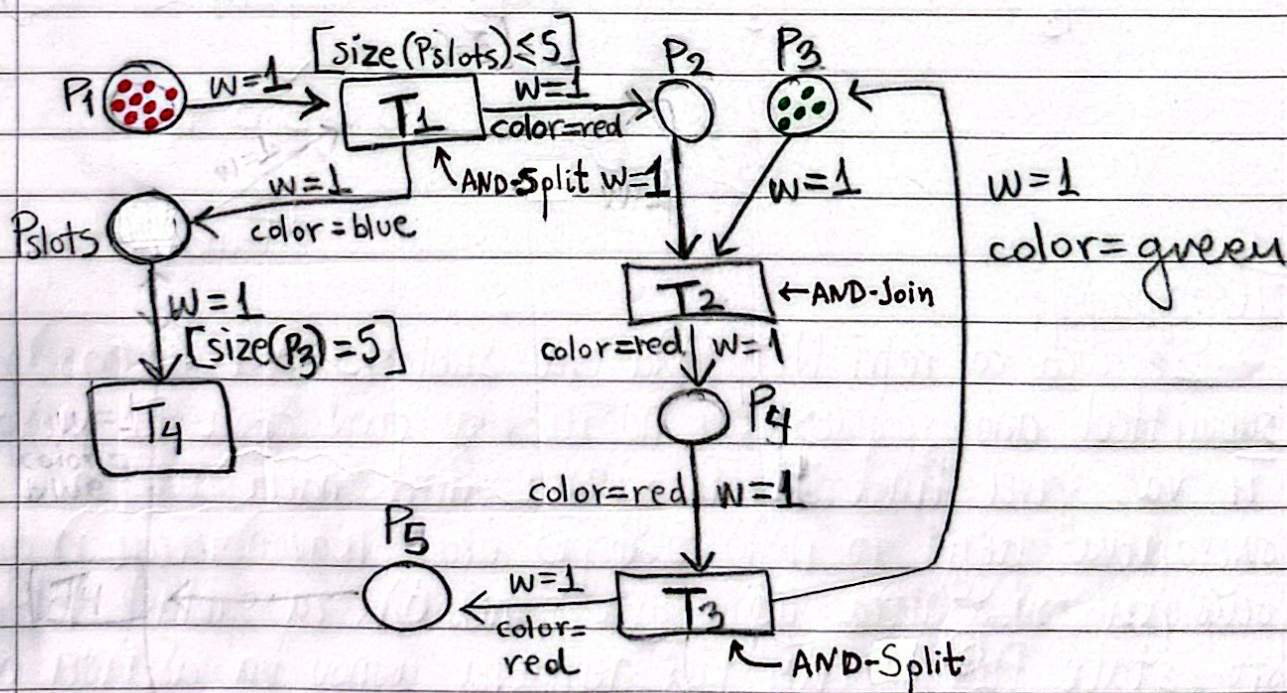
Using Colored Petri-Nets

Tokens: 9 subtasks (red), 5 resources (green),
5 slots (blue) in Pslots.

Places: $P_1 \rightarrow$ Task submitted.
 $P_2 \rightarrow$ Pending tasks for allocation, ready to run.
 $P_3 \rightarrow$ Available resources.
 $P_4 \rightarrow$ Task allocates a resource.
 $P_5 \rightarrow$ Task execution completed.
Pslots $\rightarrow$ Available slots that let subtasks to be enqueued.

Transitions: $T_1 \rightarrow$ Enqueue the task to run it.
 $T_2 \rightarrow$ Assign an available resource to a subtask.
 $T_3 \rightarrow$ Execute subtask on the resource, until execution is finished.
 $T_4 \rightarrow$ Sink transition, lets a new round begin (red color)

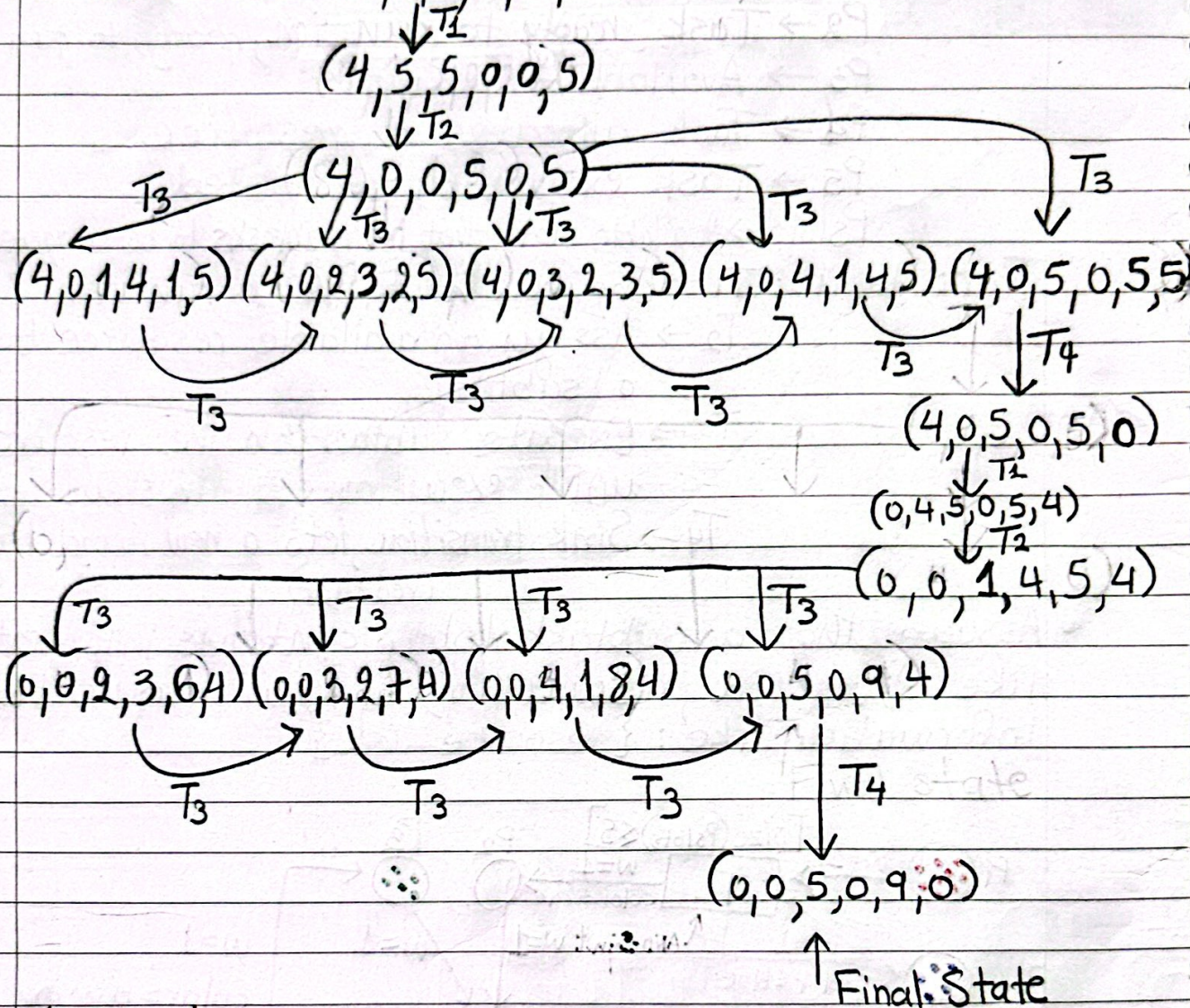
- Assume that a subtask token contains information like: {task-id, resource-id} and a resource token information like: {resource-id}.



Places/States = $(P_1, P_2, P_3, P_4, P_5, P_{slots})$

Reachability graph:

initial state: $(9, 0, 5, 0, 0, 0)$



Περιγραφή:

Σ' αυτό το Petri Net όλα τα subtasks (red tokens) γίνονται submitted από χρήστες και παραμένουν στην θέση P_2 μέχρι το transition T_1 να γίνει fired. Το T_1 γίνεται fired αυτόματα και μόνο όταν ικανοποιείται η guard συνθήκη του. Έπειτα το T_1 ως AND-Split transition, αφού έχει πάρει τα ^{κατάλληλα} subtask tokens (red color) στο state P_2 και έχει δημιουργήσει τα αντίστοιχα slot tokens (blue color) στο state P_{slots} , σταματάει και περιμένει μέχρι το P_{slots} να μην έχει κανένα slot token. Στη συνέχεια το transition T_2 , που υποθέτουμε πως είναι πιο χρήσιμο και θα γίνεται triggered πάντα πριν το T_4 , καταναλώνει αυτόματα και συνεχόμενα ένα subtask και ένα resource token (από το state P_2 και P_3 αντίστοιχα) και ως AND-Join transition παράγει κάθε φορά στο state P_4 ένα subtask token (red color), το οποίο περιέχει και ό,τι πληροφορία χρειάζεται. Έπειτα το transition P_3 καταναλώνει αυτό το παραχόμενο subtask token, κάνει execute the task, και ως AND-Split τονοθετεί ένα subtask token στα executed tasks (state P_5) και ένα resource token (green color) στο state P_3 , σηματοδοτώντας έτσι πως το συγκεκριμένο resource είναι ξανά available. Τελικά, όταν όλα τα resources έχουν απελευθερωθεί, γίνεται triggered το transition T_4 και καταναλώνοντας συνεχόμενα όλα τα tokens που έχει, το T_1 μπορεί να ξανά αρχίσει να καταναλώνει ^{subtask} tokens από το state P_1 , ώστε να κάνουμε allocate και τα τελευταία τέσσερα subtask tokens.